Reproducing Mamba at Small Scale: Selective State Spaces versus Time-Invariant SSMs and Transformers

Deep Learning – Paper Implementation Project (DL-061)

Ishita Singh - 24cs2019

Abstract

We re-implement the selective state-space (S6) block of Mamba [1] in pure PyTorch and test its central claim at small scale on a Colab T4 GPU. On a synthetic selective-copying task, the selective model reaches 82.2% sequence accuracy while an otherwise identical input-independent (LTI) model reaches 0%; a parameter-matched Transformer reaches 100% at this short sequence length. On a TinyShakespeare character-level language model (about 475K parameters), the selective and LTI models are indistinguishable (2.383 vs. 2.385 bits/char), and both beat the parameter-matched Transformer (2.514). All results are from a single seed. We therefore claim only that selection is necessary for selective copying in our setting, and that we could not show a benefit of selection on language modeling at this scale.

1 Problem and Motivation

Transformers model sequences with $O(L^2)$ attention. Linear-time alternatives based on structured state-space models (SSMs) such as S4 [2] are linear time-invariant (LTI): the same dynamics apply at every position, so they cannot decide what to remember based on content. Mamba makes the SSM parameters functions of the input and recovers efficiency with a hardware-aware scan. We ask: (i) does selectivity matter on a task designed to need it; (ii) does it help on language modeling at small scale; (iii) how does Mamba compare with a Transformer of equal parameter count?

2 Background and Method

A continuous SSM $h'(t) = Ah(t) + Bx(t),\ y(t) = Ch(t)$ is discretised with step size Δ. Assuming x is constant over each step (zero-order hold), the exact solution gives

$$\bar{A} = e^{\Delta A}, \qquad \bar{B} = (\Delta A)^{-1}\big(e^{\Delta A} - I\big)\,\Delta B \approx \Delta B, \quad (1)$$

where the approximation is the Euler form used in practice (accurate for small Δ). The recurrence is

$$h_t = \bar{A}_t h_{t-1} + \bar{B}_t x_t, \qquad y_t = C_t h_t + D x_t. \quad (2)$$

With A diagonal and negative, $\bar{A} \in (0, 1)$ element-wise, so the state decays and the update is stable. In an LTI model (Δ, B, C) are constants, so the model equals a global convolution. In S6 they are *selective*: $\Delta_t = \mathrm{softplus}(\mathrm{Lin}(x_t))$, $B_t = \mathrm{Lin}_B(x_t)$, $C_t = \mathrm{Lin}_C(x_t)$. Intuitively, B_t decides what to write, C_t what to read, and Δ_t how strongly the current input replaces the state (large Δ: reset to the current input; small Δ: ignore it). Selectivity breaks the convolutional view, but the recurrence stays associative under

$$(a_1, b_1) \circ (a_2, b_2) = (a_1 a_2,\ a_2 b_1 + b_2), \quad (3)$$

so it can be computed by a parallel prefix scan. Costs: attention is $O(L^2 D)$ in time; the SSM is $O(LDN)$ in time with $O(DN)$ state at inference.

2.1 Derivation of the discretisation

For $h' = Ah + Bx$ with x held constant over $[t, t{+}\Delta]$, the variation-of-constants formula gives

$$h(t{+}\Delta) = e^{\Delta A} h(t) + \int_0^{\Delta} e^{(\Delta - s)A} B\, ds\; x = e^{\Delta A} h(t) + A^{-1}\big(e^{\Delta A} - I\big)\, B \quad (4)$$

Hence $\bar{A} = e^{\Delta A}$ and $\bar{B} = A^{-1}(e^{\Delta A} - I)B = (\Delta A)^{-1}(e^{\Delta A} - I)\,\Delta B$. For small ΔA, $(\Delta A)^{-1}(e^{\Delta A} - I) \to I$, so $\bar{B} \approx \Delta B$. The notebook compares the two for $A = -(1, 2, 3)$: at $\Delta{=}0.01$ they agree to within 1%, at $\Delta{=}0.1$ they differ by about 5–14%, and at $\Delta{=}1$ they differ substantially ($\bar{B} = 0.63$ vs. 1.0 for the first state dimension). Since Δ is initialised in $[10^{-3}, 10^{-1}]$ and learned, we use the cheaper Euler form as the paper does.

2.2 Why the recurrence is parallelisable

Write the update as an affine map $h \mapsto a\,h + b$. Composing two such maps gives $h \mapsto a_2(a_1 h + b_1) + b_2 = (a_1 a_2)h + (a_2 b_1 + b_2)$, which is Eq. (3). Because this composition is associative, all prefixes $h_1, \ldots, h_L$ can be computed by a parallel scan with depth $O(\log L)$ instead of L sequential steps. Selectivity makes $a_t = \bar{A}_t$ and $b_t = \bar{B}_t x_t$ depend on the input, but they are still known for all t before the scan starts, so associativity is all that is needed.

3 Implementation

All code was written for this project in PyTorch (notebook `Mamba_from_scratch.ipynb`). **Block:** `in_proj` produces (x, z) of width $2D_{\text{in}}$ (expand 2); x passes through a causal depthwise convolution (kernel 4, left padding $k-1$, cropped to length L) and SiLU; (Δ, B, C) come from `x_proj` and `dt_proj` (low-rank, rank $\lceil d/16 \rceil$); the scan output is gated by $\text{SiLU}(z)$ and projected by `out_proj`. $A = -\exp(A_{\log})$ is initialised S4D-real ($A = -(1, \dots, N)$) and Δ such that $\text{softplus}(\text{bias})$ is log-uniform in $[10^{-3}, 10^{-1}]$, as in the paper.

Scan: we implement a sequential reference scan (Python loop) and a Hillis–Steele parallel scan using Eq. (3): in round k every position combines with the one 2^k steps earlier, so $h_{1:L}$ is obtained in $\lceil \log_2 L \rceil$ rounds. The parallel scan materialises tensors of shape (B, L, D_{in}, N), so it is far less memory-efficient than the paper's fused CUDA kernel. Two unit tests are in the notebook: the two scans agree (max abs. difference 9.5×10^{-7}), and the block is causal (perturbing inputs at positions ≥ 10 leaves outputs before position 10 unchanged).

Algorithm 1: selective scan (batch b, length l, channels d, state n).
Input: $u, \Delta \in \mathbb{R}^{b\times l\times d}$, $A \in \mathbb{R}^{d\times n}$, $B, C \in \mathbb{R}^{b\times l\times n}$, $D \in \mathbb{R}^{d}$
1. $\bar{A} \leftarrow \exp(\Delta \otimes A)$ $\quad (b, l, d, n)$
2. $\bar{B}u \leftarrow \Delta \otimes B \otimes u$ $\quad (b, l, d, n)$
3. *sequential:* for $t = 1..l$: $h \leftarrow \bar{A}_t \odot h + \bar{B}u_t$; $y_t \leftarrow \sum_n C_{t,n} h_{\cdot,n}$
parallel: for $o = 1, 2, 4, \cdots < l$: $x_t \leftarrow \bar{A}_t \odot x_{t-o} + x_t$, $\bar{A}_t \leftarrow \bar{A}_t \odot \bar{A}_{t-o}$ (positions $t \leq o$ unchanged); then $y_t \leftarrow \sum_n C_{t,n} x_{t,\cdot,n}$
4. **return** $y + D \odot u$

Design choices. Weight decay is applied to all parameters for simplicity (the official code exempts $A_{\log}$ and D), a small deviation from the reference implementation. We use RMSNorm before each mixer, residual connections, and tie the embedding and output head. The Transformer baseline uses learned absolute positions of the same width as the model; its attention uses a boolean causal mask. Data for both tasks is generated on the fly from fixed seeds, and evaluation uses a fixed held-out generator (selective copy: 512 fresh sequences; LM: 20 validation batches). **Models.** Both architectures share one wrapper: embedding, n layers of (RMSNorm $\rightarrow$ mixer $\rightarrow$ residual), final RMSNorm, and a weight-tied output head. The mixer is either the Mamba block or a causal pre-norm Transformer block (4 heads, GELU MLP, learned absolute positions). **LTI ablation:** Δ, B, C are replaced by input-independent learned parameters (Δ per channel, $B, C \in \mathbb{R}^N$ shared), with the same initialisation of Δ and A; all else is unchanged.

Equal size. "Equal size" means equal total parameter count. The Transformer has the same width and depth as Mamba, and its MLP width d_{ff} is chosen so that its total parameters match the selective Mamba model (parameters are linear in d_{ff}): $d_{\text{ff}} = 105$ for copying and 179 for the LM. The LTI ablation is *not* size-matched: it has about 15% (copy) and 10% (LM) fewer parameters because it has no `x_proj`.

4 Experimental Setup

Tasks. *Selective copying*: K=8 data tokens (vocabulary 8) are placed at random positions among noise tokens (token 0) in a window of L=64, followed by K recall markers; the target at the marker positions is the data tokens in order. Loss and accuracy are measured on those K positions; *sequence accuracy* requires all K correct, *token accuracy* is per token (chance level is $1/8$). *Character LM*: TinyShakespeare (1.12M characters, vocabulary 65), 90/10 train/validation split, context 128; the metric is validation bits per character (cross-entropy in nats divided by $\ln 2$).

Changes from the paper. The paper evaluates selective copying at L=4096 with 16 tokens and trains much larger language models on large corpora. We use L=64, K=8, two-layer models, and a 475K-parameter LM trained for 4000 steps, so that everything runs on a free T4. Absolute numbers are not comparable to the paper; only qualitative comparisons are meaningful.

Table 1: Training configuration (Colab T4, single run, seed 0).

	Selective copy	Char LM
d_{model} / layers / N	64 / 2 / 16	128 / 4 / 16
Context	64+8	128
Batch / steps	64 / 3000	32 / 4000
Peak lr	3×10^{-3}	2×10^{-3}
Optimizer	AdamW, wd 0.01, one-cycle (10% warmup), clip 1.0	

5 Results

Table 2: Selective copying after 3000 steps (K=8, L=64).

Model	Params	Tok. acc.	Seq. acc.	Time
Mamba	66.1K	0.976	0.822	325 s
Mamba-LTI	55.9K	0.504	0.000	325 s
Transformer	66.1K	1.000	1.000	26 s

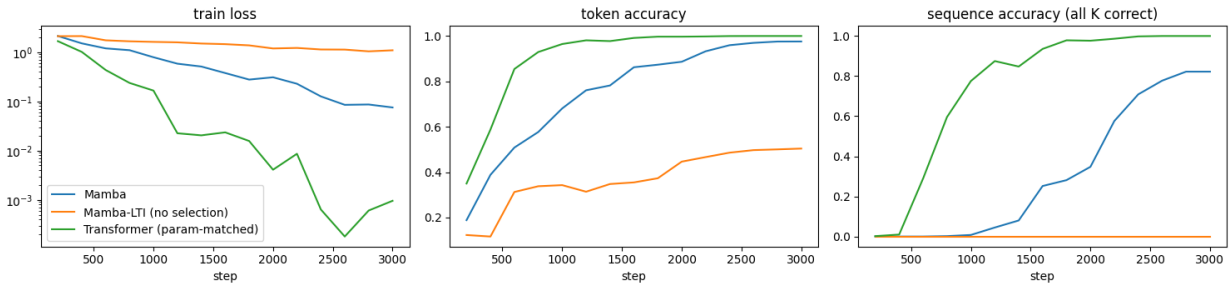


Figure 1: Selective copying (L=64, K=8): training loss (log scale), token accuracy and sequence accuracy for the three models.

Table 3: TinyShakespeare after 4000 steps (validation).

Model	Params	Bits/char	Perplexity	Time
Mamba	474.9K	2.383	5.22	1585 s
Mamba-LTI	425.9K	2.385	5.22	1577 s
Transformer	474.6K	2.514	5.71	64 s

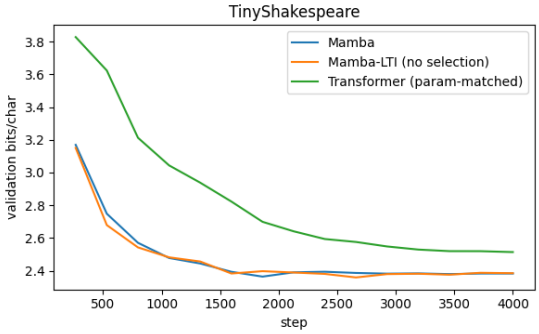


Figure 2: Validation bits/char on TinyShakespeare (lower is better).

Selective copying. Selection is clearly necessary: the LTI model never reproduces a full sequence (0.0%) and plateaus around 50% token accuracy, while the selective model reaches 82.2%. (The LTI model's partial token accuracy is well above chance, 12.5%; we did not investigate what it exploits.) The Transformer, however, solves the task completely, reaching 100% sequence accuracy by about step 2600, much faster than Mamba. Mamba's sequence accuracy rose from 0.58 at step 2200 to 0.78 at step 2600 and then flattened near 0.82 as the one-cycle learning rate decayed to zero; we do not know whether a longer schedule would close the gap to the Transformer.

Language modeling. The selective and LTI Mamba models are indistinguishable (2.383 vs. 2.385 bits/char, perplexity 5.22 for both; the curves cross repeatedly), so we cannot claim that selectivity helps here. Both Mamba variants are about 0.13 bits/char better than the parameter-matched Transformer (2.514), whose validation loss was still decreasing slowly at the end of training. Wall-clock times (Mamba about 12× slower than the Transformer on copying and 25× on the LM) reflect our Python-level parallel scan, not the paper's fused kernel, and should not be read as a property of the architectures.

6 Ablation

The ablation removes input dependence of Δ, B and C simultaneously. On selective copying this reduces sequence accuracy from 82.2% to 0% and token accuracy from 97.6% to 50.4%, consistent with the paper's argument: a time-invariant system applies the same dynamics at every position and cannot choose which tokens to keep. On the LM the same ablation changes bits/char by 0.002, i.e. no measurable effect. Caveats: the LTI model has fewer parameters, so the comparison is not size-matched (though a $\sim$15% difference seems unlikely to explain a gap from 82% to 0%); and we did not ablate Δ, B and C separately, which the paper does (finding selective Δ most important).

7 Failure Cases and Caveats

(i) *Single seed.* All numbers come from one run, so we have no estimate of run-to-run variance; differences such as 0.13 bits/char between Mamba and the Transformer should be treated as tentative. (ii) *Transformer wins at short length.* At L=64 attention can retrieve content directly and solves selective copying faster and more completely than Mamba; the paper's argument concerns much longer sequences, which we did not test. (iii) *Mamba not converged on copying.* Its accuracy was still improving until the learning rate annealed, so the reported 82.2% is likely not its ceiling; we did not test longer training. (iv) *Likely overfitting on the LM.* Mamba's validation loss reached 2.364 bits/char at step

1862, was 2.394 at step 2394, and ended at 2.383; the final single-batch training loss was about 1.2–1.3 nats (about 1.8 bits/char), well below validation. Together with the 16 passes over a 1 MB corpus (4000 steps $\times$ 32 $\times$ 128 characters) this suggests overfitting, but we did not test it, e.g. with dropout or early stopping. (v) *Not size-matched LTI.* See above. (vi) *Speed.* The scan is pure PyTorch, so no efficiency claims can be made.

8 Related Work

S4 [2] introduced stable parameterisations of LTI SSMs for long sequences. Mamba [1] adds input-dependent parameters and a hardware-aware scan and reports competitive language modeling with linear scaling in length. The selective-copying task follows the paper's synthetic benchmark. Our work is a small-scale re-implementation and ablation, not a new result.

9 Conclusion

At small scale, input-dependent selection is necessary for selective copying (82% vs. 0% sequence accuracy against an LTI ablation) but a parameter-matched Transformer still solves the task better at $L{=}64$. On TinyShakespeare, selection made no measurable difference, while both Mamba variants outperformed the matched Transformer by about 0.13 bits/char. Next steps are multiple seeds, longer sequences, separate ablations of Δ, B and C, and regularisation for the LM.

References

[1] A. Gu and T. Dao, "Mamba: Linear-time sequence modeling with selective state spaces," *COLM*, 2024, arXiv:2312.00752.

[2] A. Gu, K. Goel, and C. Ré, "Efficiently modeling long sequences with structured state spaces," *ICLR*, 2022.